

TALENTBRIDGE

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

<https://github.com/gankaiken/UM-Hackathon>

1. Introduction:

TalentBridge AI tackles the core problems in today's hiring systems. Instead of relying on basic keyword matching in resumes, it uses a multi-agent AI pipeline to create a blind evaluation layer—assessing candidates based on practical skills, reasoning, and initiative rather than just what's written on paper.

1.1. Purpose:

The system is built around an asynchronous, event-driven agent architecture that aims to **reduce human bias** in recruitment, **identify integrity issues** such as cheating, and **offer meaningful development opportunities** to all applicants. Its core purpose is to prioritize proven ability and potential over formal qualifications.

1. Architecture:

- Modular Next.js App Router with an internal "Agent Bus."

2. Data Flows:

- Unstructured Job Description (JD)
- Structured Dimensions
- Adaptive Interview
- Behavioral Sentinel
- Bias-Blind Verdict.

3. Model Process:

- [Z.AI](#) (GLM-5.1) serves as the core reasoning engine for 7 AI agents.

4. Role of Reference:

- The Job Description (JD) acts as the source-of-truth for the entire session lifecycle.

1.2. Background:

Today's hiring still leans heavily on resumes—static profiles that can be easily optimized or even generated by AI, yet don't reflect a candidate's real abilities. The result is poor hiring decisions, higher turnover, and overlooked talent, especially in fast-growing markets like Malaysia.

1. Previous Version:

- a. Current hiring practices **rely heavily on resumes** and **keyword-based filtering** systems, which often fail to reflect a candidate's true abilities. This approach makes it easy to game the system while overlooking individuals with real potential but weaker presentation. As a result, companies face **biased selection**, **inconsistent evaluation**, and higher turnover, while capable talent—especially in emerging markets—remains underrecognized.

2. Changes in Major Architectural Components

- a. Every candidate is able to take the AI interview which will analyze them based on a range of aspects coded in our model prompt.
- b. HR directly gets a summary of every candidate including their strengths and weaknesses and 'verdict cards' which give HR a quick overview and comparison of every candidate.

3. New capabilities that are only introduced in this

- a. **AI interview** which analyzes every strength and weakness from different aspect (refer to 7 AI models) with cheating detector implemented
- b. **Verdict cards** provided which greatly increase the efficiency of hiring process
- c. **Job Description Mapper** which automatically defines 5 dimensions for every candidate to be analyzed

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
HR Managers	Primary User	High-fidelity "Verdict Cards," 60% reduction in screen time.
Founders/ CXOs	Stakeholder	Data-backed hiring decisions, strong employer branding.
Candidates	End User	A fair, conversational chance to prove talent; zero-ghosting feedback.
Dev Team	Provider	Modular agent code, strict schema

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (AI-powered hiring platform)
Architecture	Architecture: Monolithic Web Application with AI Processing Pipeline

2.1.2 LLM as Service Layer

TalentBridge treats the GLM API not as a simple "text completer," but as a State Machine.

We use the API to:

- Reason: Deciding which skill to probe next based on the "Coverage Map."
- Audit: Cross-referencing Sentinel flags with linguistic style shifts.

2.1.3 Dependency Diagram

- d. How the prompts are being constructed and sent to the GLM API
- e. What goes into the context window?
- f. How is your system receiving, parsing the responses and passing to the next system component?
- g. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM
- h. Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]
- i. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]
- j. Must highlights the distributed components if the system is planning to be deployed across multiple services

2.2.2. Sequence Diagram

- a. Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]
- b. Show the interactions step by step.
- c. Do demonstration of each key system features does operate from start to finish

2.2. Technological Stack

- 2.2.1. Frontend: Next.js 15 (App Router) with Tailwind CSS v4. Animations via Motion (Framer Motion).
- 2.2.2. Backend: Serverless Node.js functions within Next.js. Streaming handled via Web Streams API.
- 2.2.3. Database: SQLite (Better-SQLite3) managed by Drizzle ORM. Synchronous, file-based for ultra-low latency in local deployments.
- 2.2.4. AI Engine: ZhipuAI (GLM-4) for multi-lingual (BM/EN/Manglish) support and JSON mode.
- 2.2.5. Cloud & Infra: Containerized via Docker, deployed on Google Cloud Run via the AI Studio platform.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.2. Data Flow Diagram (DFD)

- Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases
- Show how proper laws are met (e.g. Black Hole)

Entity/Process	Input	Output	Law Compliance
External: Recruiter	Raw JD Text	Trigger: Extract Rubric	No Black Hole: High-level text is utilized immediately by the Mapper.
P1: The Mapper	Raw JD Text	MapperResult (JSON)	Consistency: Input matches the domain logic of the Job.
External: Candidate	Resume + Chat Msg	Session Context	No Miracle: AI responses are strictly gated by Candidate input.
P2: The Sentinel	Window Events	SentinelData	Integrity: Acts as a "write-only" listener to prevent tampering.
P3: The Auditor	Full Transcript	VerdictResult	Closure: Every byte recorded in the transcript is evaluated.

2.3.3. Normalized Database Schema

- `jd_cache`: Stores extracted dimensions and weights. Unique ID serves as the "Role Key."
- `sessions`: Tracks candidate metadata, live coverage maps, the "Sentinel Data" blob, and final verdicts.
- `transcripts`: Log of every turn. crucially stores `strategist_json` for auditing the AI's own decision logic.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	JD Activator	Convert any text JD into an active interview portal.
2	Live Inquisitor	Multi-turn streaming chat that mirrors the candidate's language style.
3	Cheat-Guard (Sentinel)	Real-time monitoring of tab switches and copy-pastes
4	Verdict Generator	Visual PDF-style cards with triage (GREEN/AMBER/RED) and upskill paths

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Persistence		Sessions can be resumed if the browser closes
Integrity		Tamper-evident Sentinel logs (locked once session ends)
Latency		Character-streaming ensures < 500ms time-to-first-token for conversation
Scalability		
Reliability		
Maintainability		

Token Latency		
Cost Efficiency		

3.3. Out of Scope / Future Enhancements

- a. The Grand Vision:
- b. Voice Forensics: Using pitch and pause analysis to detect nervousness or external coaching.
- c. Company Culture "Digital Twin": Uploading team Slack/Discord logs to the Mapper to tune the "Cultural Fit" dimension.
- d. Integrations: Auto-posting Green-tier candidates directly to Workday or Greenhouse.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.2. Priority Matrix:

4.3. Monitoring:

4.4. Assumptions:

- 4.4.1. Requires ZHIPU_API_KEY for full pipeline; falls back to mockMappers for offline/demo reliability.
- 4.4.2. Assumes modern evergreen browsers (Chrome/Safari/Edge) to support visibilitychange API for Sentinel.

4.5. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output	High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]
Google Maps API	Tracking rider location and optimize the route	Medium level- if real time tracking is being disabled.

5. Project Management & Team Contributions

- 5.1. Project Timeline:** Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]
- 5.2. Team Members Role:** Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]
- 5.3. Recommendations:** If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]